# KanoonBuddy — System Architecture Reference (HLD + LLD) for PRD

Sep 30, 2026 · @Akshita

## Purpose and how to use this

This document gives the system architecture for KanoonBuddy, the LexBharat legal research product, at a level of detail you can lift straight into a PRD. It covers the high-level design (what the parts are) and the low-level design (how each part works inside).

Indian Kanoon's Prism is used here only as a reference for the feature set. Prism has not published its internals, and its tool pages sit behind a login, so nothing below is a description of their code.

What is verified and what is not:

| Claim | Status |
|---|---|
| The six Prism tools and what each one advertises | Verified from the public Prism landing page |
| KanoonBuddy runs on the open-india-law dataset with a local Qdrant | From you |
| Every architecture, schema, API and flow below | Designed here, standard practice for this kind of system, not observed from Prism |

How this maps into a PRD: the product surface section feeds your feature list, the architecture and module sections feed the technical requirements appendix, the non-functional section feeds your quality bar, and the build phases section feeds your milestones. The risks section lists what the PRD still has to decide.

## Reference teardown: the six Prism tools

Prism is six tools sitting on one legal database. Five of the six are the same retrieval-and-answer engine with a different prompt and a different output shape, which is the single most useful fact for planning build effort.

| Tool | What it advertises | What it implies technically | Engine reuse |
|---|---|---|---|
| Know Your Kanoon | Research chatbot answering legal queries with citations drawn from the Indian Kanoon database | Retrieval over a judgment corpus, answer generation, citation linking | Core engine |
| Upload and Chat | Upload a document, ask questions, get answers referenced to that document | File ingestion, OCR, per-document index, page-level citation | Core engine, private index |
| DocHub | 80+ templates, AI editing, upload and compare multiple drafts side by side | Template library, rich text editor, diff engine, export | Separate product |
| CasePredictAI | Case outcome analysis plus several strategic approaches with reasoning | Retrieval plus a fixed structured output schema | Core engine, new schema |
| Counter Argument Generator | Rebuttals, opposing viewpoints, procedural defences, supporting authorities | Retrieval plus an adversarial role prompt | Core engine, new prompt |
| Legal Sandbox | Study Buddy and moot court simulation for exam prep and practice | Multi-turn roleplay with session state, less retrieval-dependent | Core engine, session state |

The practical reading for KanoonBuddy: build one excellent retrieval engine, then four of the six tools are prompt and output work rather than new systems. DocHub is the outlier and should be scoped as its own project.

Prism also runs an accounts layer (sign up, log in) and a learning section called IKademy, so a user, plan and usage model is assumed throughout this document.

## Product surface for KanoonBuddy

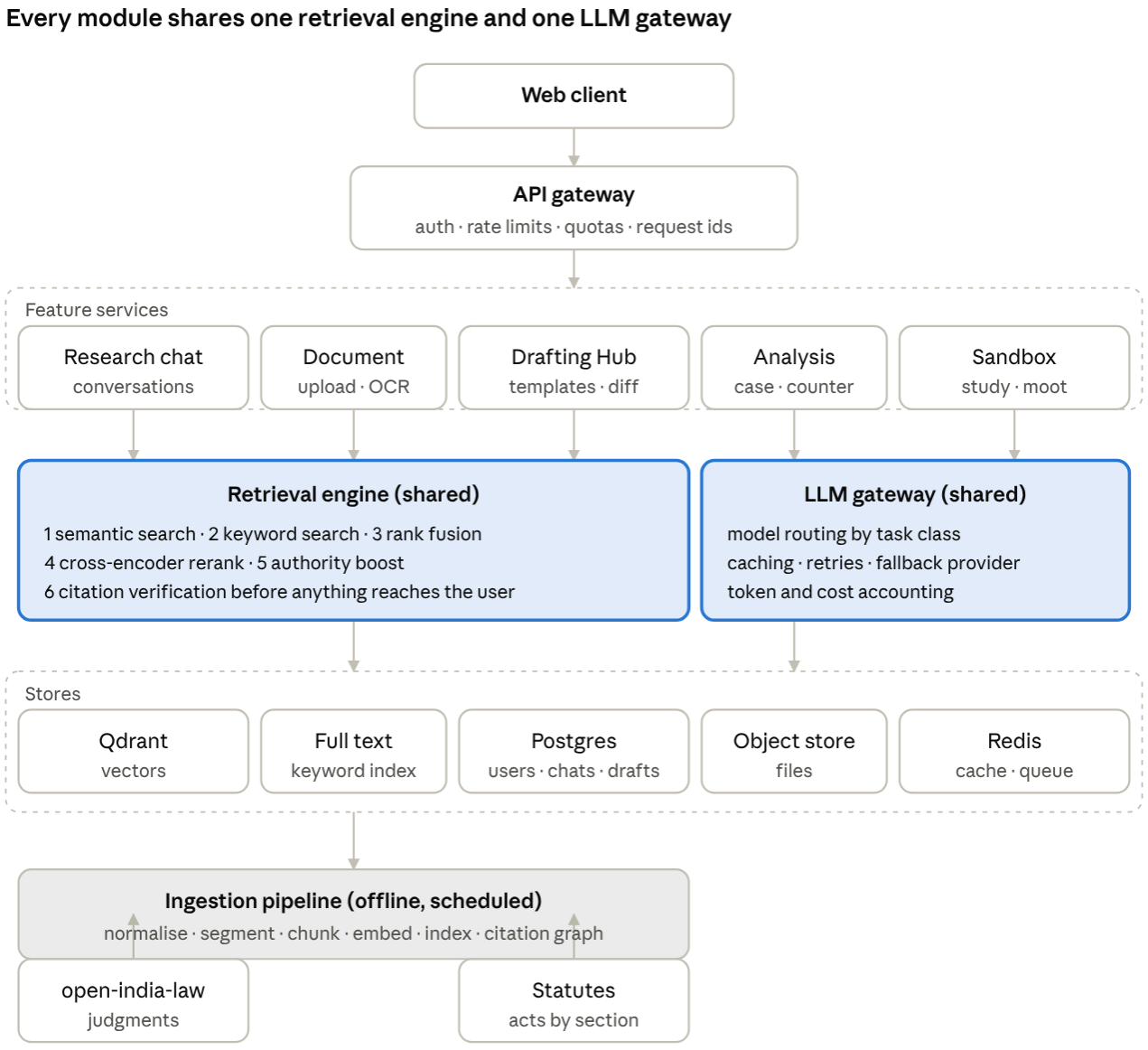
Six user-facing modules, each mapped to the services that serve it. Module names are working names for the PRD to confirm.

| Module | User does | Backend services involved | Build size |
|---|---|---|---|
| Research Chat | Asks a legal question, gets an answer with linked citations | Chat, Retrieval, LLM Gateway | Medium |
| Document Chat | Uploads a PDF or Word file, asks questions about it | Document, Retrieval, LLM Gateway, Object Storage | Medium |
| Drafting Hub | Picks a template, generates a draft, edits with AI, compares two versions | Drafting, LLM Gateway, Object Storage | Large |
| Case Analysis | Enters facts, gets issues, precedents, likely outcome, strategy options | Analysis, Retrieval, LLM Gateway | Small once engine exists |
| Counter Argument | Enters a position, gets the opposing case with authorities | Analysis, Retrieval, LLM Gateway | Small once engine exists |
| Learning Sandbox | Study plans, quizzes, moot court practice against an AI bench | Sandbox, LLM Gateway | Small to medium |

Cross-cutting surfaces that the PRD must also specify: sign up and log in, plan and quota display, conversation history, saved research, citation viewer, and an export path for every output.

The one differentiator to protect: every factual claim in an answer must carry a citation that resolves to a real judgment in the corpus. This is both the quality bar and the trust story, and it constrains the design in section 7.

# System architecture




KanoonBuddy architecture · 5 modules, 2 shared services, 5 stores

The two accented boxes are the whole engineering bet. Build the retrieval engine and the LLM gateway once, well, and each feature module above them becomes prompt and interface work rather than a new system. Requests flow down from the client; the ingestion pipeline runs offline on its own schedule and only writes to the stores.

Layer by layer:

- **Web client**: chat, document viewer, draft editor, all streaming.
- **API gateway**: authentication, rate limits, quota checks, request ids. Nothing business-specific lives here.
- **Feature services**: one per module, deliberately thin. Each assembles a prompt and calls the two shared services.

- **Retrieval engine**: the only component that touches the corpus indexes. Detailed in section 7.
- **LLM gateway**: the only component that talks to model providers. Detailed in section 13.
- **Stores**: Qdrant for vectors, a keyword index for exact matching, Postgres for application data, object storage for files, Redis for cache and queues.
- **Ingestion pipeline**: offline and scheduled, never in the request path. Detailed in section 6.

## Core request flows

### Flow A: research chat question

1. The client opens a streaming connection and posts the user's question with the conversation id.
2. The Chat service loads the last few turns plus the running summary of older turns.
3. Query rewrite: a small, cheap model turns the conversation into one standalone search query, and extracts filters it can see (court, year, statute section).
4. Hybrid retrieval: semantic search in Qdrant and keyword search run in parallel, 30 candidates each.
5. Fusion: the two ranked lists merge into one using reciprocal rank fusion.
6. Rerank: a cross-encoder scores the merged list against the question and keeps the top 8 to 10 chunks.
7. Prompt assembly: system rules, conversation summary, recent turns, the chosen chunks each tagged with a source id, and the question.
8. The strong model streams the answer back to the client token by token.
9. Citation verification runs on the finished answer: every cited source id must exist in the retrieved set and resolve to a real case id. Unverified citations are stripped and logged.
10. The message, its citations and its token counts are written to the database, and usage is incremented.

### Flow B: document upload and chat

1. The client requests an upload URL and puts the file directly into object storage.
2. The Document service creates a record with status `processing` and enqueues an ingestion job.
3. The worker extracts text. If the page has no text layer it runs OCR.

4. The worker chunks the text, keeping the page number on every chunk, embeds the chunks, and writes them into a Qdrant collection namespaced to that document.
5. Status flips to `ready` and the client is notified.
6. Questions then run Flow A, but retrieval is scoped to that document's namespace, and citations point to page numbers instead of judgments.
7. Optional: a second retrieval pass against the main corpus, so the answer can also cite case law. This is a product decision the PRD should make explicit.

### Flow C: generate a draft

1. The user picks a template. The client renders a form from the template's field schema.
2. The user fills the fields and submits.
3. The Drafting service merges fields into the template body, then asks the model to write only the clauses marked as AI-generated.
4. The assembled draft is saved as version 1 and opened in the editor.
5. Each later AI edit sends only the selected text plus the instruction, and replaces only that span. Every accepted edit creates a new version row.

### Flow D: case analysis

1. The user submits a fact pattern and the matter type.
2. The service runs retrieval twice: once on the facts, once on the legal issues the model extracts from the facts.
3. The model is asked for a strict JSON object, not prose.
4. The service validates the JSON against the schema. On failure it retries once with the validation error appended.
5. Every precedent in the JSON is checked against the corpus before the result is rendered.
6. The client renders the JSON into sections, so the layout is under product control rather than model control.

## Data layer and ingestion pipeline

The corpus is the product. Answer quality is capped by what is in the index and how well it is cut up, so this is the section to over-invest in.

## Source

KanoonBuddy runs on the open-india-law static dataset (vaquill-AI on GitHub, vaquill/open-india-law on Hugging Face), held locally. Because it is a static snapshot, the PRD must state a refresh policy: how new judgments arrive, how often, and who is responsible. A research tool that silently goes stale loses lawyers fast.

## Pipeline stages

1. **Load**: read the raw dataset files into a staging table. Keep the raw text immutable.
2. **Normalise**: strip markup, fix encoding, normalise whitespace and quotation marks, standardise court names and judge names.
3. **Extract metadata**: court, bench, date of judgment, case number, neutral citation, reported citations, parties, statutes cited, disposal type. Some will come from the dataset fields, the rest from regular expressions over the header block.
4. **Segment**: split each judgment into its structural parts. Indian judgments broadly follow headnote, facts, issues, submissions, analysis, holding, order. Regular expressions plus a small classifier per paragraph is enough.
5. **Chunk**: within each part, 400 to 800 tokens per chunk with roughly 80 tokens of overlap. Never split mid-sentence. Every chunk keeps its case id, part label and position.
6. **Embed**: batch the chunks through the embedding model. Store vectors in Qdrant with the case id, court, year and part label as payload fields, so filters work at search time.
7. **Index for keywords**: the same chunks go into a full-text index for exact matching on section numbers, party names and Latin phrases, which vector search handles badly.
8. **Build the citation graph**: parse citations inside each judgment, resolve them to case ids, and store the edges. In-degree becomes an authority score used at ranking time.

## Chunking rules that matter

- A chunk that loses its context is useless. Prepend a short header to every chunk before embedding: case title, court, year, part label. This alone lifts retrieval quality noticeably.
- Statutes are chunked by section, never by size. A section is the unit lawyers think in.
- Keep a separate short chunk for each judgment's holding or ratio. These are the highest value passages and should be retrievable on their own.

### Idempotency and reruns

Give every pipeline stage a content hash so a rerun skips unchanged records. Version the embedding model in the chunk record. When you change models you re-embed, and you need to know which chunks are on which version while the migration runs.

## Retrieval and ranking engine

This is the shared component every module calls. Treat it as one internal service with one interface, not as code copied into each feature.

### Interface

`retrieve(query, filters, namespace, k)` returns a ranked list of passages, each with its text, its case id, its part label and a stable source id used for citation.

### The four stages

1. **Semantic search** in Qdrant. Top 30. Good at meaning and paraphrase, weak on exact identifiers.
2. **Keyword search** over the full-text index. Top 30. Good at "Section 138 NI Act", party names, and rare terms.
3. **Fusion** with reciprocal rank fusion. Score each document as the sum over both lists of 1 divided by (60 plus its rank). No tuning needed and it beats naive score-averaging because the two systems produce scores on different scales.
4. **Rerank** the fused top 40 with a cross-encoder, keep the top 8 to 10. This stage is what separates a demo from a product.

### Filters the UI must expose

Court, year range, statute or section, and judgment part. Lawyers filter constantly, and filters also reduce token cost because they shrink the candidate pool.

### Authority boost

After reranking, apply a small boost for Supreme Court over High Court over lower courts, and for citation in-degree. Keep the boost small and configurable. A large boost buries the recent, on-point High Court case that actually answers the question.

### Citation verification

The non-negotiable step. After generation:

- Parse every citation marker out of the answer.

- Each marker must map to a source id that was in the retrieved set for this turn.
- Each source id must resolve to a live case record.
- Anything failing either check is removed from the answer and written to a `hallucinated_citation` log with the query, so you can measure the rate.

The log is a product metric, not just debugging. Target it explicitly in the PRD, for example under 0.5 percent of answers after verification.

### Evaluation harness

Build this with the engine, not after. A fixed set of 100 to 200 real legal questions with known correct authorities, run on every change, scored on recall at 10 and on whether the right case appears in the final context. Without it you cannot tell whether a prompt change helped or hurt.

## Research chat service

### Conversation state

A conversation is a row; every turn is a message row. Context sent to the model is the last 6 turns in full plus a rolling summary of everything before them. The summary is regenerated by the cheap model whenever the conversation passes a turn threshold, and stored on the conversation row so it is not recomputed per request.

### Streaming

Use server-sent events. The client shows tokens as they arrive. Citations stream as a separate event at the end, after verification, so the user never sees a citation that is later stripped.

### Prompt structure

Four fixed blocks, in this order: system rules, conversation context, retrieved passages, user question. The rules block must state:

- Answer only from the passages provided.
- Cite the source id after every proposition that comes from a passage.
- If the passages do not answer the question, say so and say what is missing. Do not fall back on general knowledge.
- This is information, not legal advice.

The third rule is the one most teams skip, and it is what stops confident wrong answers on gaps in the corpus.

### What the user sees per answer

Answer text, inline citation markers, a source panel listing each cited judgment with court, year and a link to the full text, and the filters that were applied. Showing the applied filters matters because it explains why a case the user expected is absent.

### Follow-up handling

Short follow-ups like "what about limitation?" carry no retrievable content on their own. The query-rewrite step is what makes them work, and it must be evaluated separately from the main answer, because a bad rewrite fails silently.

### Regeneration and history

Every message stores the retrieved source ids and the model version used. This makes answers reproducible, lets you re-run old questions against a new pipeline during evaluation, and is what you need when a user disputes an answer.

## Document upload and chat

### Upload path

The file goes from the browser straight to object storage using a pre-signed URL. It never passes through the API server. This keeps request timeouts and memory use out of the problem, and it is the only sane path for the 200-page documents lawyers actually upload.

Accepted: PDF, DOCX, TXT, and images of documents. Cap the file size and the page count, and state both in the PRD.

### Processing job

Asynchronous, queued, with visible status. The states are `uploaded`, `extracting`, `indexing`, `ready`, `failed`, and the client polls or listens for the change.

Text extraction has two paths. A PDF with a text layer is parsed directly. A scanned PDF has no text layer and needs OCR, which is slower and costs more. Detect which one you have by sampling a few pages for extractable characters, and tell the user when OCR is running, because the wait is noticeably longer.

OCR for Indian court documents must handle Devanagari and regional scripts, not just English. Test this early with real scanned orders.

### Indexing

One Qdrant collection per document, or one shared collection with a mandatory `document_id` filter. The second is cheaper to operate; the first is cleaner for deletion. Pick one and write it into the PRD, because it affects the deletion guarantee below.

Every chunk keeps its page number and its character offsets, so a citation can jump to the exact place in the viewer.

### Answering

Retrieval is scoped to the document. Citations are page references. The viewer should highlight the cited passage when the user clicks a citation, which requires those offsets and is the feature that makes this module feel trustworthy.

### Comparison with the corpus

Decide explicitly whether document questions may also retrieve from the case law corpus. Doing it well means two retrieval calls and a prompt that labels which passages came from which source, so the user can tell apart "your contract says" from "the law says".

### Deletion and retention

Uploaded documents are client material. Support hard deletion of the file, its extracted text, and its vectors, and give a retention default. State the default in the product, not just in the terms of service.

## Drafting Hub

The largest module and the one least like the others. It is a document editor with AI inside it, so most of the work is editor engineering rather than retrieval.

### Template library

Each template is a record with a field schema and a body. The field schema drives an auto-generated form, so adding a template is content work, not code work. That property is what lets you reach 80-plus templates without 80 builds.

A template body mixes three kinds of content: fixed text, field placeholders, and clauses marked for AI generation with a short instruction attached. Templates also carry a jurisdiction and a stamp-duty or registration note where relevant.

Seed set to scope in the PRD, roughly in order of demand: rental and lease agreements, employment offer and appointment letters, non-disclosure agreements, vendor and service agreements, legal notices under common statutes, affidavits, powers of attorney, consumer complaints, bail applications, and partnership deeds.

### Generation

Fields merge deterministically. Only the AI-marked clauses go to the model, each with its own small prompt containing the field values it may use. Generating clause by clause rather than whole-document keeps output stable, keeps cost down, and means one bad clause does not require regenerating the document.

### AI editing

The user selects text and gives an instruction. The request carries the selected span, a window of surrounding text for context, and the instruction. The model returns replacement text only. The editor shows it as a suggestion the user accepts or rejects; it never overwrites silently.

Fixed instruction presets cover most use: make it stricter, make it plainer, shorten, add a termination clause, shift the balance toward one party.

### Review

Run the draft through a checklist prompt: missing clauses for this document type, undefined terms, internal inconsistencies, blank placeholders, dates that do not line up. Output is a list of flags with severity and the span each one points at, rendered as margin comments rather than prose.

### Compare

Extract text from both documents, align them, and run a text diff to get added, removed and changed spans. Then pass only the changed spans to the model to explain what each change does legally and who it favours. Running the whole document through the model instead is expensive and produces vaguer results.

### Versioning and export

Every save writes a version row. Users must be able to see the history, restore a version, and export to DOCX and PDF with formatting intact. DOCX export is a hard requirement; lawyers finish their work in Word.

### Editor choice

Use an established rich-text framework, TipTap or ProseMirror, with a schema restricted to what legal documents need: headings, numbered clauses with automatic renumbering, nested sub-clauses, tables, signature blocks. Automatic clause renumbering is the detail users notice immediately when it is missing.

## Case analysis and counter argument

Both modules are the retrieval engine plus a strict output schema. Asking the model for JSON rather than prose is what makes the output renderable, comparable across runs, and testable.

### Case analysis output schema

```
{
  "matter_type": "string",
  "issues": [{ "issue": "string", "governing_law": "string" }],
  "precedents": [{ "case_id": "string", "title": "string", "holding":
"string", "why_relevant": "string", "favours": "applicant|respondent|neutral"
}],
  "assessment": { "direction": "favourable|mixed|unfavourable", "confidence":
"low|medium|high", "reasoning": "string" },
  "strategies": [{ "approach": "string", "steps": ["string"], "risks":
["string"], "typical_timeline": "string" }],
  "missing_facts": ["string"],
  "procedural_notes": ["string"]
}
```

Two fields carry most of the product value. `missing_facts` tells the user what would change the answer, which is what a senior lawyer would say first. `confidence` must be driven by retrieval quality, not by the model's tone: when the top reranked scores are weak, force it to low.

### Counter argument output schema

```
{
  "position_summary": "string",
  "substantive_rebuttals": [{ "argument": "string", "authorities":
["case_id"], "strength": "strong|moderate|weak" }],
  "procedural_defences": [{ "defence": "string", "basis": "string" }],
  "factual_challenges": ["string"],
  "distinguishing_precedents": [{ "case_id": "string", "how_to_distinguish":
"string" }],
  "weakest_point": "string"
}
```

`distinguishing_precedents` is the most lawyerly output in the product: taking a case the other side relies on and showing why it does not apply. It needs a retrieval pass aimed at the opposing position, not just at the user's own.

### Retrieval differences

Case analysis retrieves on the facts and on the extracted issues, then filters toward the same court level where possible, because procedure varies. Counter argument runs a deliberately adversarial query: the model first states the opposing position, and that statement becomes the search query.

### Framing and safety

Call it analysis, not prediction, in the interface. Show the precedents that drove the assessment so the user can check the reasoning. Add a standing line that this is not legal advice, and never produce a numeric probability of winning, which is both unsupportable from this data and a liability.

## Learning Sandbox

The cheapest module to build and the one with the clearest audience: law students and exam candidates. It is also a low-cost acquisition channel into the paid research product.

### Study mode

A tutor persona with a stored goal, for example a specific entrance exam or paper. The service keeps a syllabus record per user and generates study plans, explanations, practice questions and flashcards against it. Progress, weak topics and attempted questions are stored so later sessions adapt rather than starting cold.

Keep generated questions in a table with their answers and explanations. Reusing them cuts cost sharply and lets you build a question bank over time as a real asset.

### Moot court simulation

This is the only module that needs an explicit state machine, because a moot has fixed stages and the AI plays different roles in each.

States: `setup` (problem, side, bench composition) → `opening` (user argues, bench listens) → `argument` (user submits points) → `questioning` (bench interrupts with questions) → `rebuttal` → `feedback` (structured scoring and comments) → `closed`.

Each turn stores the state, the speaker, and the content. The prompt for each state is different, and transitions are explicit rather than left to the model. Letting the model manage its own stage is the usual failure mode here: it drifts into being helpful instead of being a bench.

Feedback at the end should be structured, not prose: clarity of submission, use of authority, response to questions, court craft, each with a short comment.

### Retrieval in this module

Lighter than elsewhere. Retrieval matters when the bench asks about authorities the user cited, and when generating a moot problem grounded in a real line of cases. Otherwise the persona prompts do the work.

## LLM gateway and cost control

Every model call in the system goes through one internal service. No feature calls a model provider directly. This is the layer that decides whether the product can be priced below the incumbents, so it belongs in the PRD as a requirement, not as an implementation detail.

### What it does

- **Routing**: chooses the model for the task class, not for the feature.
- **Caching**: exact-match and semantic caching of repeated queries.
- **Accounting**: logs tokens in, tokens out, latency, cost and cache status per call, tagged with user, feature and model.
- **Resilience**: retries with backoff, timeouts, and a fallback model when a provider fails.
- **Guardrails**: enforces per-user and per-plan quotas before the call is made.

### Task classes and routing

| Task class | Example | Model tier | Share of calls |
|---|---|---|---|
| Utility | Query rewrite, conversation titles, intent classification, summarisation of old turns | Small | High |
| Structured | Metadata extraction, JSON analysis output, checklist review | Mid | Medium |
| Generative | Final research answers, clause drafting, counter arguments | Large | Low |
| Embedding | Indexing and search-time embedding | Embedding model | Very high volume, very low unit cost |

Most calls in a working system are utility calls. Routing them to a small model instead of the large one is the single biggest cost reduction available, usually larger than any prompt shortening.

### Caching

Three layers, cheapest first: exact-match on the normalised query, semantic match above a high similarity threshold on the rewritten query, and provider-side prompt caching for the fixed system block. Legal research has heavy query overlap, because thousands of users ask about the same well-known provisions.

### Quotas

Quotas are enforced here because this is the only place that knows the true cost. Define a free tier in messages per day, a paid tier with a higher ceiling, and a hard stop rather than a silent degradation. Display remaining quota in the interface.

### Cost observability

The metric to put on the dashboard from day one is cost per answered question, broken down by feature. Without it, pricing decisions in the PRD are guesses, and a prompt change that doubles cost goes unnoticed for weeks.

## Data model

Postgres for everything relational, Qdrant for vectors, object storage for files. Ids are UUIDs. Every table carries `created_at` and `updated_at`.

## Corpus

| Table | Key fields |
|---|---|
| `cases` | id, title, court, bench, decision_date, case_number, neutral_citation, reported_citations[], disposal, source_url, raw_text_path |
| `case_chunks` | id, case_id, part_label, position, text, token_count, embedding_model_version, vector_id |
| `statutes` | id, act_name, act_year, section_number, section_title, text |
| `case_citations` | citing_case_id, cited_case_id, treatment (followed, distinguished, overruled, referred) |
| `case_statutes` | case_id, statute_id |

## Accounts and usage

| Table | Key fields |
|---|---|
| `users` | id, email, password_hash, name, role (advocate, student, in-house, other), plan, status |
| `subscriptions` | id, user_id, plan, period_start, period_end, provider_ref |
| `usage_daily` | user_id, date, feature, requests, tokens_in, tokens_out, cost_estimate |
| `api_keys` | id, user_id, hashed_key, scopes, last_used_at |

## Conversations

| Table | Key fields |
|---|---|
| `conversations` | id, user_id, module, title, rolling_summary, message_count |
| `messages` | id, conversation_id, role, content, model, tokens_in, tokens_out, latency_ms |
| `message_sources` | message_id, source_id, case_id, chunk_id, rank, score, verified |

| Table | Key fields |
|---|---|
| `feedback` | message_id, user_id, rating, reason, comment |

`message_sources` is what makes citations auditable and what the evaluation harness reads. It is not optional.

## Documents

| Table | Key fields |
|---|---|
| `documents` | id, user_id, filename, mime_type, size_bytes, page_count, status, storage_path, ocr_used, expires_at |
| `document_chunks` | id, document_id, page_no, char_start, char_end, text, vector_id |

## Drafting

| Table | Key fields |
|---|---|
| `templates` | id, name, category, jurisdiction, field_schema (jsonb), body, version, is_active |
| `drafts` | id, user_id, template_id, title, current_version, status |
| `draft_versions` | id, draft_id, version_no, content (jsonb), created_by (user or ai), change_note |
| `draft_reviews` | id, draft_id, version_no, flags (jsonb) |
| `draft_comparisons` | id, user_id, left_ref, right_ref, diff (jsonb), analysis (jsonb) |

## Sandbox

| Table | Key fields |
|---|---|
| `sandbox_sessions` | id, user_id, mode, state, config (jsonb) |
| `sandbox_turns` | id, session_id, turn_no, speaker, content |

| Table | Key fields |
|---|---|
| `question_bank` | id, topic, difficulty, question, options (jsonb), answer, explanation |

### Qdrant collections

| Collection | Payload fields used as filters |
|---|---|
| `case_chunks` | case_id, court, year, part_label, statute_refs |
| `statute_sections` | act_name, act_year, section_number |
| `user_documents` | document_id, user_id, page_no |

## API surface

REST with JSON, except chat responses which stream over server-sent events. All routes under `/api/v1`. Authentication is a bearer token.

| Method and path | Purpose | Notes |
|---|---|---|
| `POST /auth/signup`, `/auth/login`, `/auth/refresh` | Accounts | Standard |
| `GET /me` | Profile, plan, remaining quota | Drives the quota display |
| `GET /search` | Direct corpus search without generation | Query, filters, pagination; cheap and useful on its own |
| `GET /cases/{id}` | Full judgment with metadata | Backs every citation link |
| `POST /conversations` | Start a conversation | Takes the module name |
| `GET /conversations` | List with pagination | |
| `GET /conversations/{id}` | Full message history with sources | |
| `POST /conversations/{id}/messages` | Ask a question | Streams; returns citations at the end |

| Method and path | Purpose | Notes |
|---|---|---|
| `POST /documents/uploads` | Request a pre-signed upload URL | Returns URL and document id |
| `GET /documents/{id}` | Status and metadata | Poll until `ready` |
| `POST /documents/{id}/conversations` | Start a chat scoped to that document | |
| `DELETE /documents/{id}` | Hard delete file, text and vectors | Must be a real delete |
| `GET /templates` | Template list with categories | Filterable by jurisdiction |
| `GET /templates/{id}` | Field schema for form rendering | |
| `POST /drafts` | Generate a draft from template plus fields | Returns draft id and version 1 |
| `PATCH /drafts/{id}` | Save editor content | Creates a version row |
| `POST /drafts/{id}/ai-edit` | Edit a selected span by instruction | Returns suggestion, does not apply it |
| `POST /drafts/{id}/review` | Run the checklist | Returns flags with spans |
| `POST /drafts/compare` | Compare two drafts or uploads | Returns diff plus analysis |
| `POST /drafts/{id}/export` | DOCX or PDF | |
| `POST /analysis/case` | Case analysis | Returns the JSON schema in section 11 |
| `POST /analysis/counter-argument` | Counter argument | Returns its schema |
| `POST /sandbox/sessions` | Start study or moot session | |
| `POST /sandbox/sessions/{id}/turns` | Submit a turn | Returns the next state |
| `POST /feedback` | Thumbs and comment on a message | Feeds evaluation |

Conventions to fix in the PRD: cursor pagination everywhere, a single error envelope with a machine-readable code, request ids returned on every response for support, and rate limits expressed in headers.

# Non-functional requirements

## Performance targets

| Measure | Target |
|---|---|
| First token of a research answer | Under 3 seconds |
| Complete research answer | Under 20 seconds |
| Retrieval stage alone | Under 800 ms |
| Direct search results | Under 500 ms |
| Document ready, text PDF under 50 pages | Under 60 seconds |
| Document ready, scanned PDF under 50 pages | Under 5 minutes |
| Draft generation | Under 45 seconds |

Time to first token matters more than total time. A stream that starts quickly reads as fast even when it finishes in fifteen seconds.

## Security

- Encrypt in transit and at rest. Uploaded documents get their own encryption key scope.
- Store passwords with a modern hash. Offer two-factor for paid accounts.
- Every query to Qdrant and to document tables carries a user id filter enforced server-side. A missing filter is a cross-tenant data leak, so it belongs in a shared data-access layer rather than in each feature.
- Never put uploaded document text into prompts destined for a provider that trains on inputs. Confirm and document the no-training terms with whichever model provider you choose.
- Audit log for logins, document uploads, deletions and exports.

## Privacy

Lawyers upload privileged client material. Commit to a retention default, a real deletion path, and a statement of which third parties see the text. Indian DPDP Act obligations apply, including purpose limitation and a grievance contact. This is a sales requirement as much as a legal one; firms will ask before they buy.

### Reliability

Stateless API servers behind a load balancer. Workers and queues separate from the request path, so a slow OCR job cannot affect chat. A model provider outage falls back to the secondary provider through the gateway. Target 99.5 percent monthly availability for the API at launch.

### Scale assumptions to validate

Write the numbers down so capacity can be checked: expected corpus size in judgments and chunks, concurrent users at peak, questions per active user per day, documents uploaded per day, and average pages per document. Vector memory and OCR throughput are the two things that break first.

### Observability

Structured logs with a request id through every hop. Traces across retrieve, rerank and generate, since "it was slow" is almost always one of those three. Dashboards for cost per question, cache hit rate, unverified-citation rate, retrieval recall on the evaluation set, and thumbs-down rate by feature.

## Tech stack

Chosen for a small team shipping fast, with the cheaper option named where one exists.

| Layer | Recommendation | Why | Cheaper or simpler alternative |
|---|---|---|---|
| Frontend | Next.js with TypeScript and Tailwind | Streaming support, good editor ecosystem | Plain React with Vite |
| Editor | TipTap on ProseMirror | Clause numbering, comments, suggestions | Quill, but expect to fight it on numbering |
| Backend | FastAPI on Python | Same language as the pipeline and the model SDKs | Node with Fastify if the team is stronger in TypeScript |
| Vector store | Qdrant, already in use | Filters on payload, runs locally, scales to managed later | pgvector, one less system to run |

| Layer | Recommendation | Why | Cheaper or simpler alternative |
|---|---|---|---|
| Keyword search | OpenSearch | Proper BM25 and analyzers | Postgres full-text search to start |
| Relational | Postgres | Everything else lives here too | |
| Queue and workers | Redis with Celery or RQ | Ingestion, OCR, exports | |
| Object storage | S3-compatible | Pre-signed uploads | |
| Cache | Redis | Query cache, sessions, rate limits | |
| Auth | Supabase Auth or Clerk | Do not build this | |
| Embeddings | A multilingual model with strong retrieval scores | Indian judgments carry Hindi and regional text | An open model self-hosted, cheaper at volume |
| Reranker | Cross-encoder, hosted or self-run | Largest single quality gain | bge-reranker, self-hosted |
| Generation | Two providers behind the gateway | Fallback and price negotiation | |
| OCR | A cloud OCR for accuracy, Tesseract for cost | Accuracy on Indian scans varies a lot | Benchmark both on real documents before choosing |
| Payments | Razorpay | Indian cards, UPI, GST invoicing | |
| Deployment | Containers on a managed platform | | |

Two decisions worth making deliberately. Qdrant versus pgvector: you already run Qdrant, so keep it, but be aware pgvector removes a moving part if scale stays modest. Self-hosted versus hosted embeddings: hosted is faster to launch, self-hosted wins once you re-embed the corpus repeatedly, which you will during tuning.

# Risks, assumptions and open questions

## Risks

| Risk | Impact | What reduces it |
|---|---|---|
| Corpus goes stale because the dataset is a static snapshot | High. Lawyers stop trusting it within one quarter | A refresh pipeline with a named owner, and a visible "corpus current to" date in the interface |
| Invented or misattributed citations | Fatal to credibility | Mandatory citation verification, measured and reported |
| Weak retrieval on statute and section queries | Users conclude the product cannot find basic law | Hybrid search with real keyword matching, evaluated separately from semantic |
| Model cost per answer exceeds price per user | Unit economics break at scale | Gateway routing, caching, quotas, cost dashboard from day one |
| Licensing or terms issues with the dataset | Could force a corpus change late | Get the dataset licence reviewed before build starts |
| Confidentiality of uploaded documents | Blocks firm-level sales | No-training provider terms, encryption, real deletion, a published policy |
| Drafting Hub consumes the roadmap | The research product ships late or thin | Scope it as a separate phase with its own milestone |
| OCR quality on Indian scanned orders | A whole document class becomes unusable | Benchmark OCR options on real scans in phase 1 |

## Assumptions the PRD should confirm

- The open-india-law dataset covers the courts and the date range the target users need.
- Users accept answers grounded in reported judgments only, with no access to unreported or pending matters.
- English is the primary interface language at launch, with regional language support deferred.

- The initial buyer is an individual advocate or small firm, not an enterprise procurement process.

### Open questions

- [ ] What is the corpus refresh cadence, and who owns it?
- [ ] Does document chat also cite case law, or only the uploaded document?
- [ ] Which two model providers, and on what terms regarding training on inputs?
- [ ] Free tier limits and paid price points, set against measured cost per answer
- [ ] How many templates ship at launch, and which categories
- [ ] Retention default for uploaded documents
- [ ] Is there a team or firm tier with shared workspaces at launch, or later?
- [ ] Which single metric defines launch quality: retrieval recall, verified-citation rate, or user thumbs-up rate?

## Glossary

| Term | Plain meaning |
|---|---|
| HLD | High-level design. The map of the system: what the big parts are and how they talk |
| LLD | Low-level design. Inside each part: tables, functions, rules, formats |
| RAG | Retrieval-augmented generation. Find the right documents first, then let the model answer using only those |
| Embedding | A list of numbers representing a piece of text's meaning, so similar meanings sit close together |
| Vector database | A store built to find the closest embeddings quickly. Qdrant is one |
| Chunk | A small passage a long document is cut into, because whole judgments do not fit in a prompt |
| BM25 | The classic keyword scoring method. Good at exact words a vector search misses |
| Hybrid search | Running keyword and vector search together and merging the results |

| Term | Plain meaning |
|---|---|
| Reciprocal rank fusion | A simple way to merge two ranked lists using positions rather than scores |
| Reranker | A slower, more accurate model that re-orders a shortlist against the question |
| Cross-encoder | The model type used for reranking. It reads question and passage together, which is why it is better and slower |
| Recall at 10 | Out of the right answers that exist, how many appear in the top 10 results |
| Hallucination | The model stating something the sources do not support. For citations, fatal |
| Query rewrite | Turning a conversational follow-up into a full standalone search query |
| Context window | How much text a model can read in one call |
| Token | Roughly three-quarters of a word. Cost and limits are counted in these |
| Prompt caching | Reusing an unchanged block of prompt so you are not billed full price for it again |
| SSE | Server-sent events. How the answer streams word by word to the browser |
| OCR | Reading text out of a picture of a page, needed for scanned documents |
| Namespace | A partition of the index so one user's document search cannot touch another's |
| Idempotent | Safe to run twice. Rerunning produces the same result rather than duplicates |
| Pre-signed URL | A temporary link letting the browser upload straight to storage, bypassing your server |